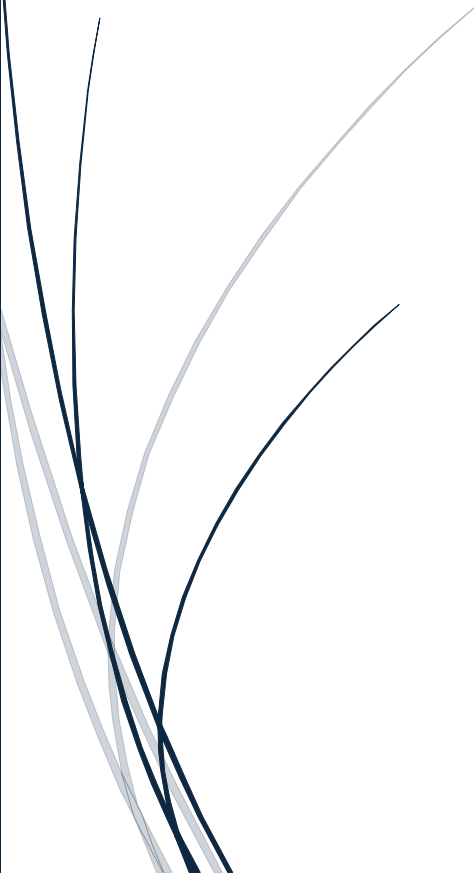


8/19/2026

CREATION OF INTELLIGENT BUG DIAGNOSIS PLATFORM WITH FIX RECOMMENDATION ASSISTANCE GROUP

Project Report



Harshitha Bejjipuram

TABLE OF CONTENTS

1. Internship Overview
2. Project Overview
3. Abstract
4. Introduction
5. Problem Statement
6. Objectives
7. Existing System
8. Proposed System
9. System Architecture
10. Technology Stack
11. System Workflow
12. Milestone 1 – RAG and Semantic Retrieval
13. Milestone 2 – Multi-Agent Diagnostic Layer
14. Milestone 3 – AI Bug Triage and Deep Log Analysis
15. Milestone 4 – Defect Pattern Analytics and Knowledge Base Growth
16. Implementation Details
17. Major Modules
18. Testing Methodology
19. Test Results
20. Challenges and Solutions
21. Learning Outcomes
22. Advantages
23. Limitations
24. Future Scope
25. Conclusion
26. Reference

1. INTERNSHIP OVERVIEW

The Infosys Springboard Virtual Internship provides learners with opportunities to gain practical exposure through guided, project-based learning. The program is designed to help learners develop technical skills and understand real-world development practices through simulated industry projects and mentorship.

As part of the internship, the project “Creation of Intelligent Bug Diagnosis Platform with Fix Recommendation Assistance Group” was developed to explore the application of Artificial Intelligence in software defect analysis.

The project focuses on combining semantic retrieval, vector search, multi-agent processing, log analysis, and knowledge-base growth into an integrated software debugging platform.

2. PROJECT OVERVIEW

Project Title

Creation of Intelligent Bug Diagnosis Platform with Fix Recommendation Assistance Group

Domain

Artificial Intelligence and Software Engineering

Core Technologies

- React.js
- Vite
- Tailwind CSS
- Node.js
- Express.js
- Python
- Fast API
- Sentence Transformers
- FAISS
- Lang Graph
- MongoDB / JSON fallback
- Docker
- Git / GitHub

Project Goal

The primary goal of Bug Sense AI is to assist developers in diagnosing software defects by automatically retrieving similar historical bugs, performing intelligent bug triage, analyzing logs, identifying probable root causes, and recommending possible fixes.

The system also supports continuous knowledge-base growth by allowing verified resolved bugs to be added to the semantic knowledge base.

3. ABSTRACT

Software debugging is an essential activity in software development, but manually investigating defects can be time-consuming and repetitive. Developers often need to examine bug descriptions, application logs, previous defect reports, and historical solutions before identifying the probable cause of a problem.

Bug Sense AI is an AI-powered software bug diagnosis and fix recommendation platform designed to reduce this manual effort.

The platform uses Retrieval-Augmented Generation (RAG), semantic similarity search, FAISS vector indexing, and a Lang Graph-based multi-agent workflow to analyze software defects.

When a developer submits a bug, the system processes the input and generates a semantic embedding using a Sentence Transformers model. FAISS performs similarity search against historical bug knowledge. Relevant historical defects are retrieved and used as context for further diagnosis.

The multi-agent layer performs specialized tasks including bug triage and deep log analysis. The system identifies severity, priority, affected module, failing class or method, error information, and probable failure reason. These results are used to determine a probable root cause and provide a fix recommendation.

The final milestone introduced a resolution verification mechanism and a knowledge-base growth process. Once a developer verifies a resolved bug, the system generates an embedding and compares it with existing FAISS knowledge. Depending on semantic similarity, the system either updates existing knowledge or creates a new knowledge entry.

The final validation was performed across 12 defect types, achieving the reported 100% diagnosis accuracy, 100% precision, 100% recall, and 100% F1-score. The reported average agent confidence was 85.4%, with an average execution latency of approximately 22.9 ms in the validation environment.

4. INTRODUCTION

Modern software systems generate a large number of defects during development and production. These defects can originate from programming errors, incorrect configurations, dependency problems, database failures, runtime exceptions, and other factors.

When a bug is reported, developers typically need to perform several activities:

- Understand the bug description.
- Search for similar historical issues.
- Analyze application logs.
- Identify the affected module.
- Determine severity and priority.
- Find the probable root cause.
- Develop and verify a solution.

Performing these tasks manually can consume significant development time.

Artificial Intelligence provides techniques that can assist in automating these activities. Semantic embeddings can identify conceptually similar bug reports even when their wording differs. Vector databases such as FAISS can provide efficient similarity retrieval. Multi-agent architectures can divide complex diagnosis into specialized tasks.

Bug Sense AI combines these technologies to create an intelligent debugging assistance platform.

5. PROBLEM STATEMENT

Traditional bug diagnosis has several limitations:

1. Developers spend significant time manually investigating defects.
2. Similar historical bugs may be overlooked.
3. Duplicate issues may be investigated repeatedly.
4. Application logs can contain large amounts of information that are difficult to analyze manually.
5. Severity and priority classification can be inconsistent.
6. Historical debugging knowledge is not always efficiently reused.
7. Newly resolved bugs may not automatically contribute to future diagnosis.

Therefore, there is a need for an intelligent system that can retrieve historical knowledge, analyze new defects, assist with root-cause identification, recommend fixes, and continuously improve its knowledge base.

6. OBJECTIVES

The major objectives of Bug Sense AI are:

- To develop an AI-powered software bug diagnosis platform.
- To implement semantic retrieval of historical bugs.
- To identify highly similar and potentially duplicate defects.
- To automate bug severity and priority classification.
- To identify the affected software module.
- To perform deep analysis of application logs.
- To identify probable root causes.
- To provide fix recommendations.
- To implement a multi-agent diagnostic workflow.
- To provide resolution verification.
- To continuously grow the knowledge base.
- To identify recurring defect patterns through analytics.

7. EXISTING SYSTEM

In a conventional debugging environment, the workflow generally follows:

Bug Report → Manual Investigation → Historical Search → Log Analysis → Root Cause Identification → Fix Development → Verification

The process depends heavily on developer experience.

Limitations

Manual Investigation

Developers need to inspect and understand each issue individually.

Repeated Analysis

Similar bugs may require repeated investigation if historical solutions are difficult to locate.

Log Complexity

Large application logs can make it difficult to quickly identify the relevant failure information.

Knowledge Reuse

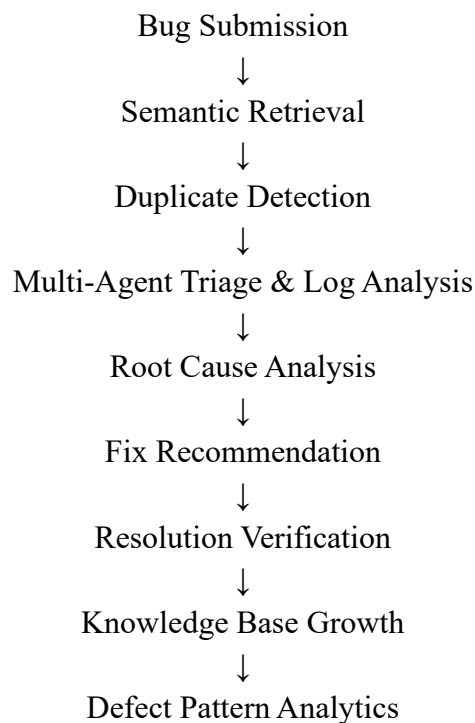
Previously resolved bugs may not be effectively reused for new defects.

Limited Automation

Conventional systems primarily provide issue tracking rather than complete AI-assisted diagnosis.

8. PROPOSED SYSTEM

Bug Sense AI introduces an intelligent workflow:



The proposed system retrieves relevant historical information before performing detailed diagnosis.

This allows previous debugging knowledge to become part of the diagnostic process.

9. SYSTEM ARCHITECTURE

The platform follows a modular three-layer architecture.

9.1 Frontend Layer

The frontend is developed using:

- React.js

- Vite
- Tailwind CSS
- Axios

The frontend provides:

- Bug submission
- Dashboard
- Diagnosis results
- Historical information
- Resolution verification
- Knowledge-base statistics
- Analytics

9.2 Backend Layer

The backend uses:

- Node.js
- Express.js
- Winston
- Multer

The Express server acts as the API gateway between the frontend and Python AI service.

9.3 AI Layer

The AI service uses:

- Python
- Fast API
- Sentence Transformers
- FAISS
- Lang Graph
- NumPy

This layer handles:

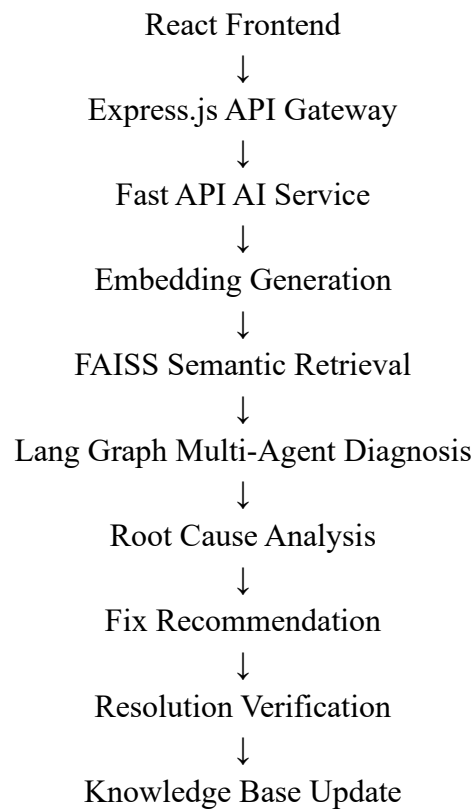
- Embedding generation
- Semantic retrieval
- Similarity search
- Duplicate detection
- Multi-agent diagnosis
- Knowledge-base updates

9.4 Data Layer

MongoDB is used for application-related data, with JSON fallback where applicable.

FAISS maintains the semantic vector index and associated metadata.

Architecture Flow



10. TECHNOLOGY STACK

Layer	Technology
Frontend	React.js
Build Tool	Vite
Styling	Tailwind CSS
HTTP	Axios
Backend	Node.js
API Framework	Express.js
AI Service	Python
AI API Framework	Fast API
Embeddings	Sentence Transformers
Vector Search	FAISS
Agent Orchestration	Lang Graph
Data Processing	NumPy, pandas
PDF Processing	PyPDF2
DOCX Processing	python-docx
Database	MongoDB / JSON fallback
Logging	Winston
Containerization	Docker
Version Control	Git / GitHub

11. SYSTEM WORKFLOW

The complete system workflow is:

Step 1 – Bug Submission

The developer submits a software bug through the dashboard.

Step 2 – Input Processing

The submitted bug description and available logs are processed.

Step 3 – Embedding Generation

The system converts the bug information into a semantic vector representation.

Step 4 – Semantic Retrieval

FAISS searches the existing knowledge base for similar historical defects.

Step 5 – Duplicate Detection

Similarity scores are evaluated to identify highly similar issues.

Step 6 – Multi-Agent Diagnosis

The bug is processed through the Lang Graph-based diagnostic workflow.

Step 7 – Triage

The system predicts:

- Severity
- Priority
- Affected module

Step 8 – Log Analysis

Relevant technical information is extracted from logs.

Step 9 – Root Cause Analysis

Retrieved historical knowledge and agent results are used to determine the probable root cause.

Step 10 – Fix Recommendation

The system provides a recommendation to assist the developer.

Step 11 – Resolution Verification

After implementing the fix, the developer verifies the resolution through the dashboard.

Step 12 – Knowledge Base Update

The verified resolution is converted into an embedding and compared with existing knowledge.

Step 13 – Analytics Update

The updated knowledge base contributes to future retrieval and defect pattern analysis

12. MILESTONE 1 – RAG AND SEMANTIC RETRIEVAL

The first milestone focused on developing the knowledge retrieval foundation.

12.1 Data Ingestion

The system supports ingestion of historical defect information from:

- PDF files
- DOCX files
- CSV files
- Logs

The information is processed and converted into suitable text representations.

12.2 Embedding Generation

The Sentence Transformers model **all-MiniLM-L6-v2** is used to generate semantic embeddings.

This allows the system to represent the meaning of bug reports as numerical vectors.

12.3 FAISS Similarity Search

The generated embeddings are stored in a FAISS index.

When a new bug is submitted, its embedding is compared against stored vectors to retrieve relevant historical defects.

12.4 Duplicate Detection

A similarity threshold of approximately 90% is used for identifying highly similar knowledge.

This helps reduce repeated investigation of similar defects.

13. MILESTONE 2 – MULTI-AGENT DIAGNOSTIC LAYER

Milestone 2 introduced a multi-agent architecture using Lang Graph.

The diagnostic process is divided into specialized tasks.

13.1 Triage Agent

The triage agent analyzes the submitted defect and determines:

- Severity
- Priority
- Affected module

- Initial classification

13.2 Log Analysis Agent

The log analysis agent examines available application logs.

It extracts information such as:

- Error messages
- Failure reasons
- Failing classes
- Failing methods
- Timestamps

13.3 Orchestrator

The Lang Graph workflow coordinates the different agents.

This creates a structured diagnostic process rather than relying on a single analysis component.

14. MILESTONE 3 – AI BUG TRIAGE AND DEEP LOG ANALYSIS

Milestone 3 enhanced the diagnostic capabilities of the platform.

14.1 Severity Classification

The system determines the severity of a reported defect based on its impact.

14.2 Priority Escalation

The system uses a P1–P4 priority model:

Priority Description

P1 Critical

P2 High

P3 Medium

P4 Low

14.3 Module Identification

The system identifies the software module associated with the defect.

This helps developers understand where the problem is likely to be located.

14.4 Explainable Classification

The system provides:

- Severity _ reason
- Priority _ reason

along with the classification.

This makes the AI output more understandable to developers.

14.5 Deep Log Analysis

The system extracts:

- Failing class
- Failing method
- Timestamp
- Error message
- Failure reason

These details support the root-cause analysis stage.

15. MILESTONE 4 – DEFECT PATTERN ANALYTICS AND KNOWLEDGE BASE GROWTH

Milestone 4 focused on continuous improvement of the platform.

15.1 Defect Pattern Analytics

The analytics component identifies recurring defect patterns and provides information that can help understand frequently occurring issues.

15.2 Resolution Verification

A dedicated resolution verification component was implemented in the dashboard.

The workflow is:

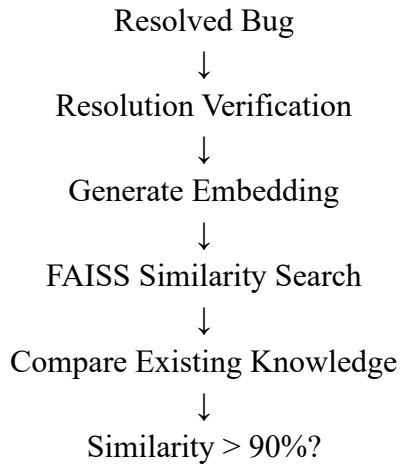
AI Diagnosis → Developer Implements Fix → Developer Reviews Result → Mark as Resolved & Verify

This ensures that only verified resolutions are considered for knowledge-base growth.

15.3 Knowledge Base Growth

The verified bug is submitted to the knowledge update mechanism.

The process is:



If the similarity exceeds the configured threshold, existing knowledge can be updated.

Otherwise, a new knowledge entry is created.

Knowledge Growth Mechanism



This mechanism enables the system to continuously expand its debugging knowledge.

16. IMPLEMENTATION DETAILS

16.1 Frontend

The frontend was implemented using React.js and Vite.

The dashboard provides interfaces for:

- Bug submission
- AI analysis
- Results visualization

- Resolution verification
- Knowledge statistics
- Defect analytics

16.2 Express Backend

The Express backend provides API routes for communicating with the frontend and AI service.

Major endpoints include:

POST /api/bugs/resolve

Used for resolution verification.

POST /api/knowledge/add

Used to add verified knowledge.

GET /api/knowledge/stats

Used to retrieve knowledge-base statistics.

16.3 FastAPI AI Service

The Python service handles the AI processing pipeline.

Responsibilities include:

- Embedding generation
- FAISS search
- Similarity calculation
- Multi-agent processing
- Knowledge updates

16.4 FAISS Storage

The vector index is persisted using:

- Faiss_index.bin
- faiss_metadata.json

This allows the knowledge base to remain available between application executions.

17. MAJOR MODULES

Module 1 – Bug Submission

Allows developers to submit defect information.

Module 2 – Semantic Retrieval

Retrieves relevant historical bugs using vector similarity.

Module 3 – Duplicate Detection

Identifies highly similar historical defects.

Module 4 – AI Triage

Predicts severity, priority, and module.

Module 5 – Log Analysis

Extracts technical information from application logs.

Module 6 – Root Cause Analysis

Uses retrieved context and diagnostic results to identify probable causes.

Module 7 – Fix Recommendation

Provides a recommended resolution approach.

Module 8 – Resolution Verification

Allows developers to confirm that the proposed fix has been successfully implemented.

Module 9 – Knowledge Base Growth

Adds verified resolved bugs to the semantic knowledge base.

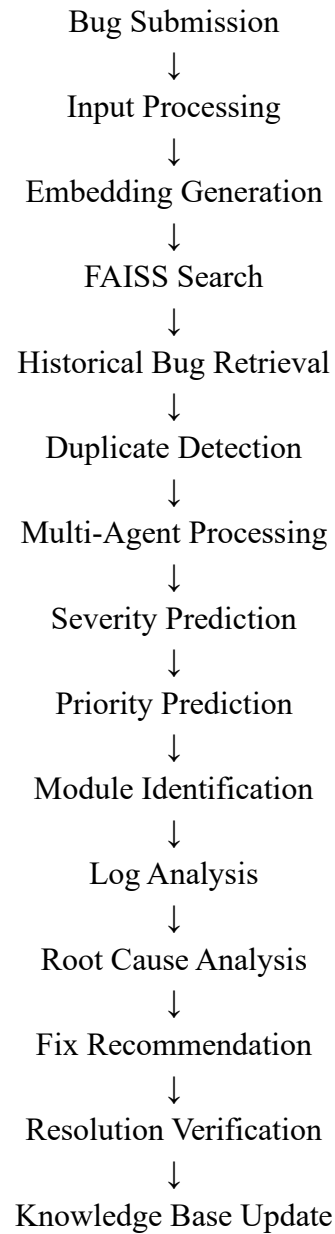
Module 10 – Defect Pattern Analytics

Provides information about recurring defect patterns.

18. TESTING METHODOLOGY

The system was tested using an end-to-end validation workflow.

Test Workflow



The testing covered multiple defect categories to validate the complete diagnostic pipeline.

19. TEST RESULTS

The final automated validation covered 12 defect types.

Metric	Result
Diagnosis Accuracy	100% (12/12)
Precision	100%

Recall	100%
F1-Score	100%
Average Agent Confidence	85.4%
Average Execution Latency	22.9 ms
Knowledge Growth – Created	12
Knowledge Growth – Updated	0

The results demonstrate successful processing of the tested defect categories in the validation environment.

The knowledge-base mechanism also successfully created new entries for the tested resolved defects where the similarity condition did not result in an update to an existing entry.

20. CHALLENGES AND SOLUTIONS

Challenge 1 – Duplicate Bug Detection

Problem

Similar bugs may use different wording.

Solution

Semantic embeddings and FAISS similarity search were implemented to compare the meaning of bug reports rather than relying only on exact text matching.

Challenge 2 – Large Log Information

Problem

Application logs can contain large amounts of information.

Solution

A dedicated log-analysis agent was implemented to extract relevant technical information such as errors, failing classes, methods, timestamps, and failure reasons.

Challenge 3 – Knowledge Base Growth

Problem

Resolved bugs should not be added blindly to the knowledge base.

Solution

A resolution verification step was introduced before updating the knowledge base.

Challenge 4 – FAISS Statistics

Problem

Knowledge-base statistics initially did not correctly reflect the FAISS index.

Solution

The knowledge statistics endpoint was updated to query the actual FAISS-based knowledge information.

Challenge 5 – Backend Integration

Problem

The frontend, Express server, and Python AI service needed coordinated communication.

Solution

Dedicated API routes and service endpoints were implemented to connect the application layers.

21. LEARNING OUTCOMES

The project provided practical exposure to several areas of software and AI development.

Technical Learning

- Retrieval-Augmented Generation
- Semantic embeddings
- FAISS vector search
- Sentence Transformers
- Multi-agent architecture
- Lang Graph
- Fast API
- Express.js
- React.js
- MongoDB
- Docker
- Git and GitHub

Software Engineering Learning

The project improved understanding of:

- Modular architecture
- API integration
- Backend–frontend communication
- Error handling
- Testing
- Debugging
- Version control
- Documentation

Professional Learning

The project also provided experience in:

- Working with project milestones
- Understanding requirements
- Implementing incremental features
- Presenting technical work
- Receiving mentor feedback
- Testing and validating an end-to-end system

22. ADVANTAGES

Bug Sense AI provides the following advantages:

1. Reduces manual debugging effort.
2. Retrieves relevant historical defects.
3. Helps identify duplicate issues.
4. Automates bug triage.
5. Provides severity and priority reasoning.
6. Performs structured log analysis.
7. Assists with root-cause identification.

8. Provides fix recommendations.
9. Supports resolution verification.
10. Continuously grows the knowledge base.
11. Provides defect-pattern analytics.
12. Uses a modular multi-agent architecture.

23. LIMITATIONS

The current system has some limitations:

- Recommendation quality depends on the quality of historical knowledge.
- AI-generated diagnoses require developer validation.
- Completely new and highly complex defects may require additional investigation.
- Source-code-level analysis is not fully automated.
- Performance can depend on the size and quality of the knowledge base.
- The current implementation primarily uses bug descriptions and available logs as diagnostic inputs.

24. FUTURE SCOPE

The platform can be extended with:

Automated Source-Code Analysis

The system can be enhanced to analyze source code along with bug reports and logs.

GitHub Integration

Future versions can integrate directly with repositories and issue trackers.

CI/CD Integration

Bug detection and diagnosis can be incorporated into continuous integration and deployment pipelines.

Automated Pull Requests

Future implementations could generate suggested code changes for developer review.

Advanced AI Agents

Additional specialized agents can be introduced for:

- Security analysis
- Performance analysis
- Database diagnosis
- Dependency analysis
- Code-quality analysis

Real-Time Monitoring

The platform can be extended to monitor application logs in real time and automatically identify emerging defects.

Advanced Analytics

More advanced dashboards can visualize:

- Recurring defect patterns
- Module-wise defect distribution
- Severity trends
- Resolution time
- Historical defect frequency

25. CONCLUSION

The Bug Sense AI project demonstrates how Artificial Intelligence can be applied to assist software developers in diagnosing and resolving software defects.

The platform integrates semantic retrieval, FAISS vector search, duplicate detection, multi-agent diagnosis, bug triage, deep log analysis, root-cause analysis, fix recommendation, resolution verification, and knowledge-base growth into a unified workflow.

The project was developed progressively through four milestones.

Milestone 1 established the RAG and semantic retrieval foundation.

Milestone 2 introduced the Lang Graph-based multi-agent diagnostic layer.

Milestone 3 enhanced AI-based bug triage and deep log analysis.

Milestone 4 introduced defect pattern analytics, resolution verification, and continuous knowledge-base growth.

The final validation across 12 defect types achieved the reported 100% diagnosis accuracy, precision, recall, and F1-score, with an average agent confidence of 85.4% and average execution latency of approximately 22.9 ms in the validation environment.

The knowledge-base growth mechanism further improves the platform by allowing verified resolutions to become reusable knowledge for future bug diagnosis.

Overall, the project provided practical experience in AI engineering, full-stack development, vector search, multi-agent systems, API integration, testing, and software engineering practices. It also demonstrated how an AI-assisted debugging platform can support developers by reducing repetitive analysis and making historical defect knowledge easier to reuse.

26. REFERENCES

1. Infosys Springboard – Official Infosys digital learning platform.
2. Infosys Springboard Virtual Internship Program information.
3. Sentence Transformers documentation and related research on semantic embeddings.
4. FAISS – Facebook AI Similarity Search documentation.
5. Lang Graph documentation for stateful and multi-agent workflows.
6. Fast API documentation.
7. React.js documentation.
8. Node.js and Express.js documentation.
9. MongoDB documentation.
10. Research literature related to Retrieval-Augmented Generation and AI-assisted software debugging.

APPENDIX A – PROJECT SCREENSHOTS

The following screenshots should be inserted from the actual implemented application:

Figure 1: Bug Sense AI Dashboard

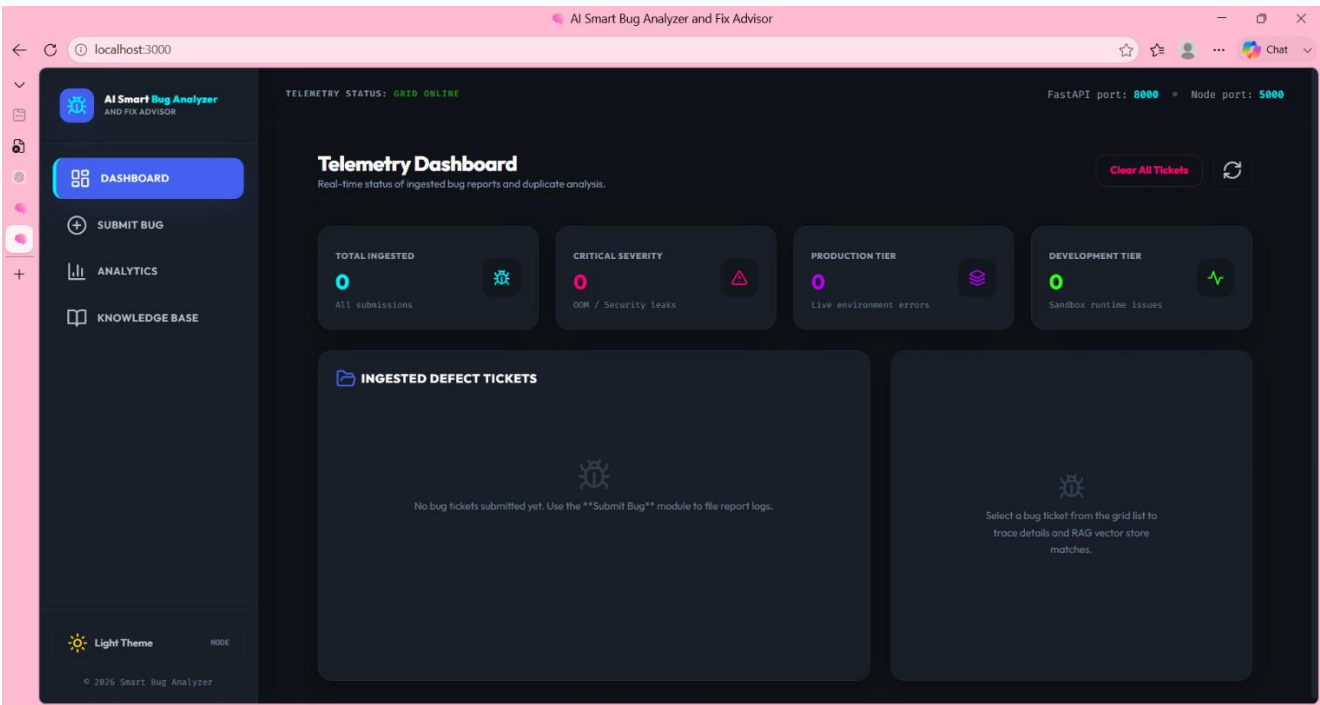


Figure 2: Bug Submission Interface

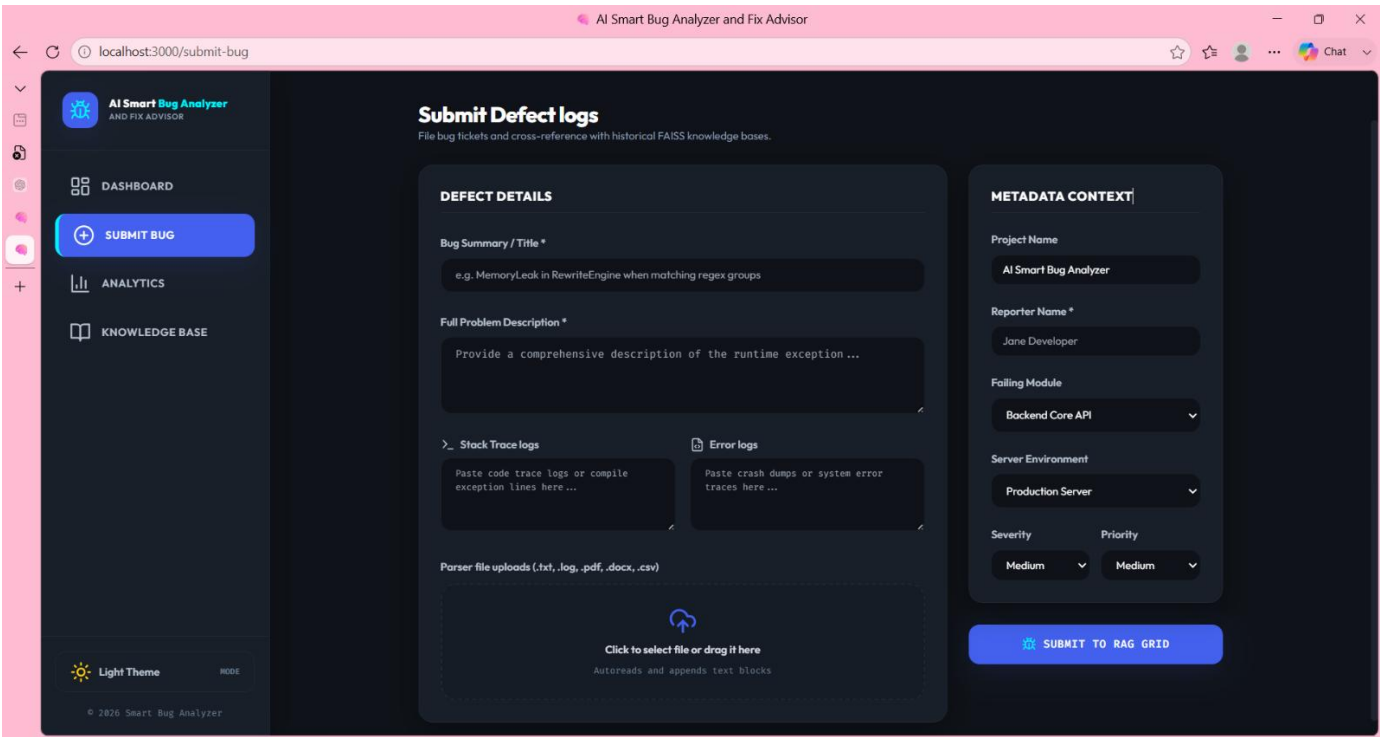


Figure 3: AI Bug Diagnosis Result

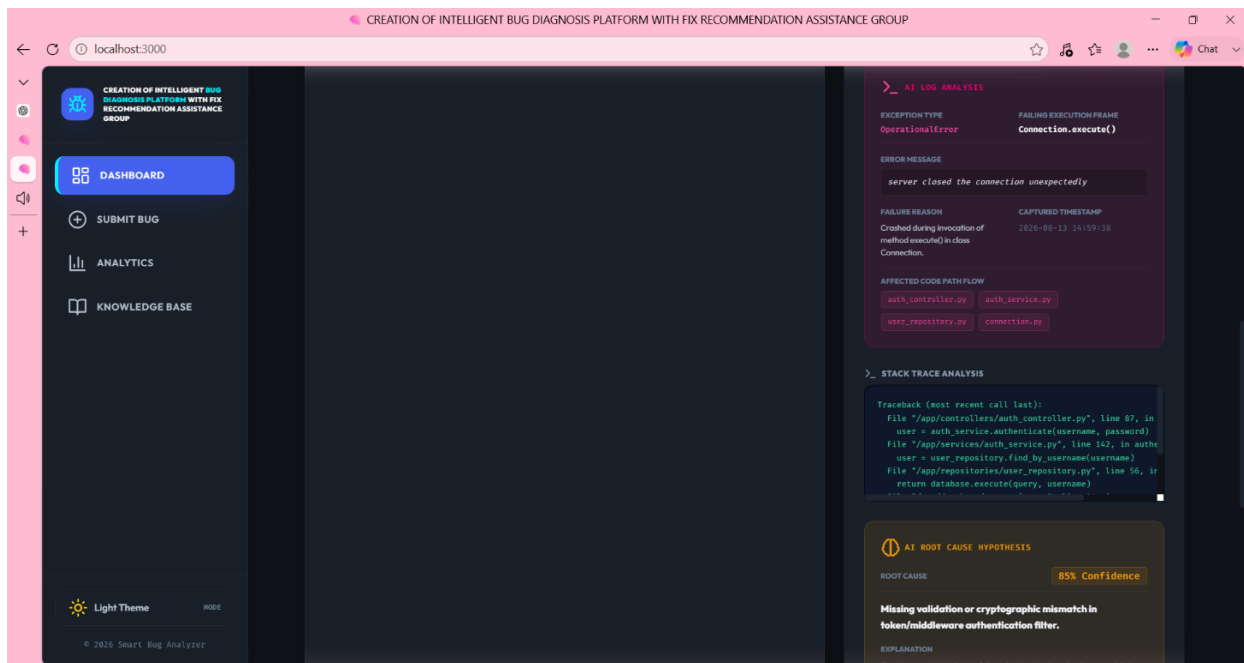


Figure 4: Semantic Retrieval / Historical Bug Results

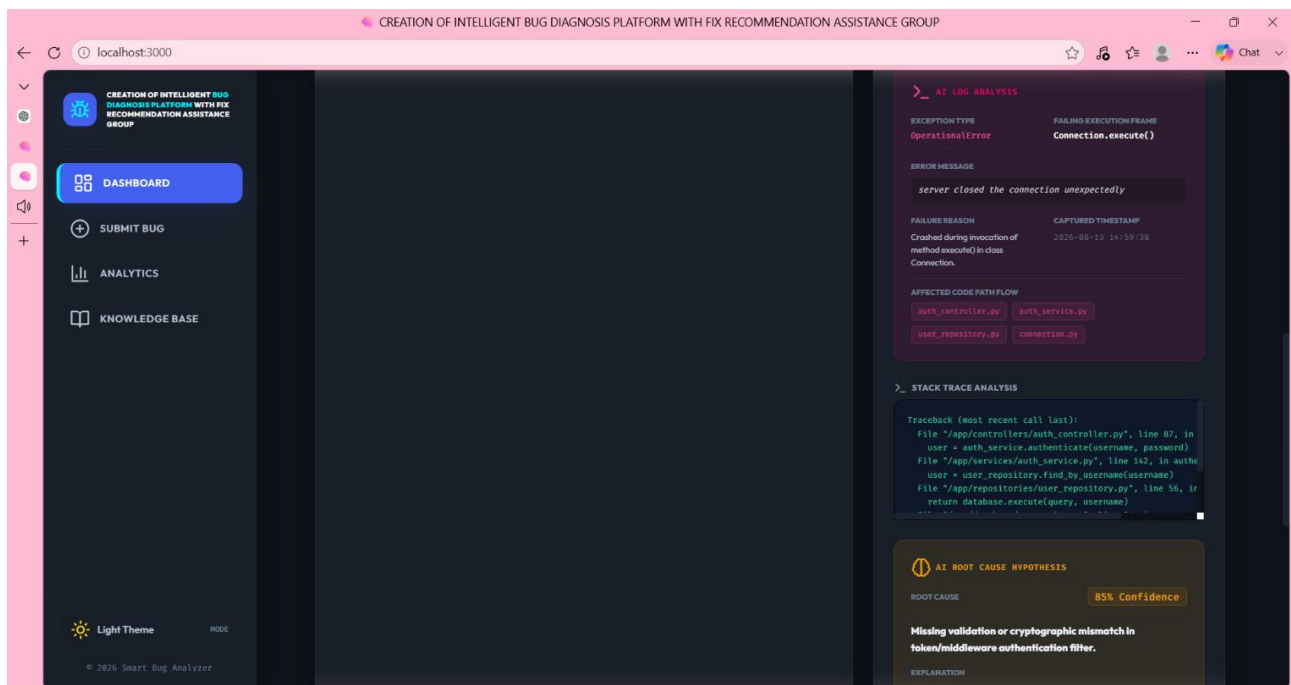


Figure 5: Severity, Priority and Module Prediction

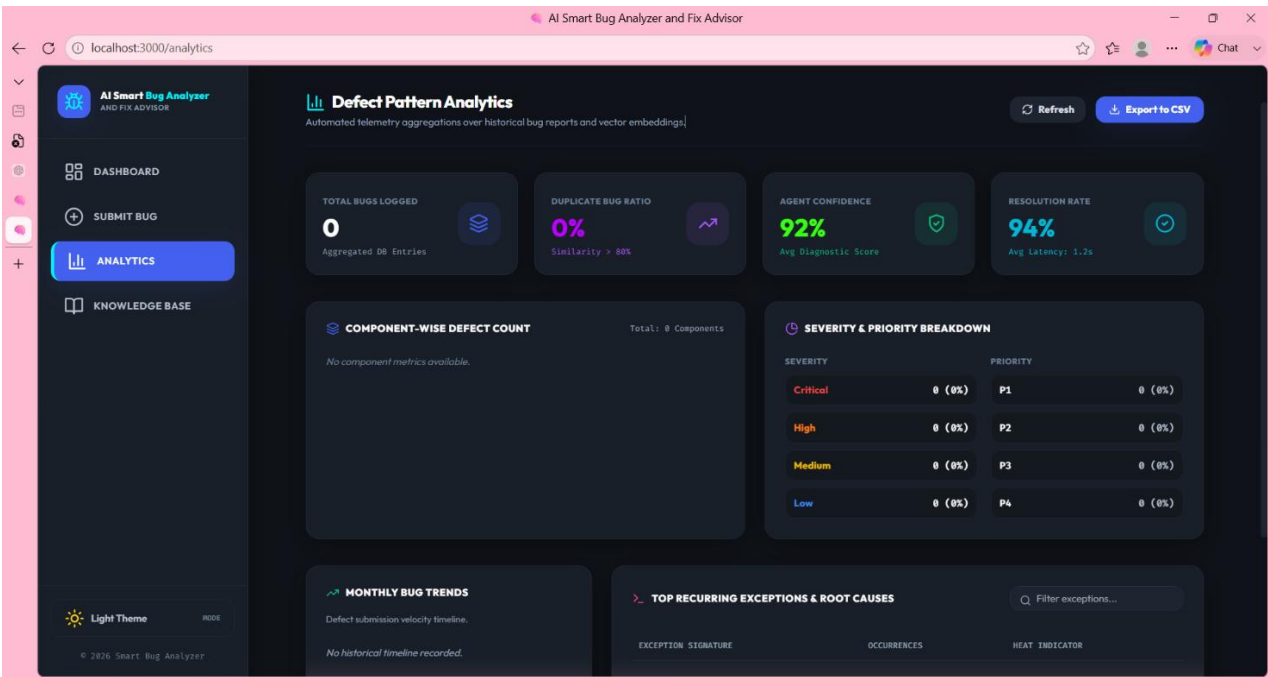


Figure 6: Deep Log Analysis Result

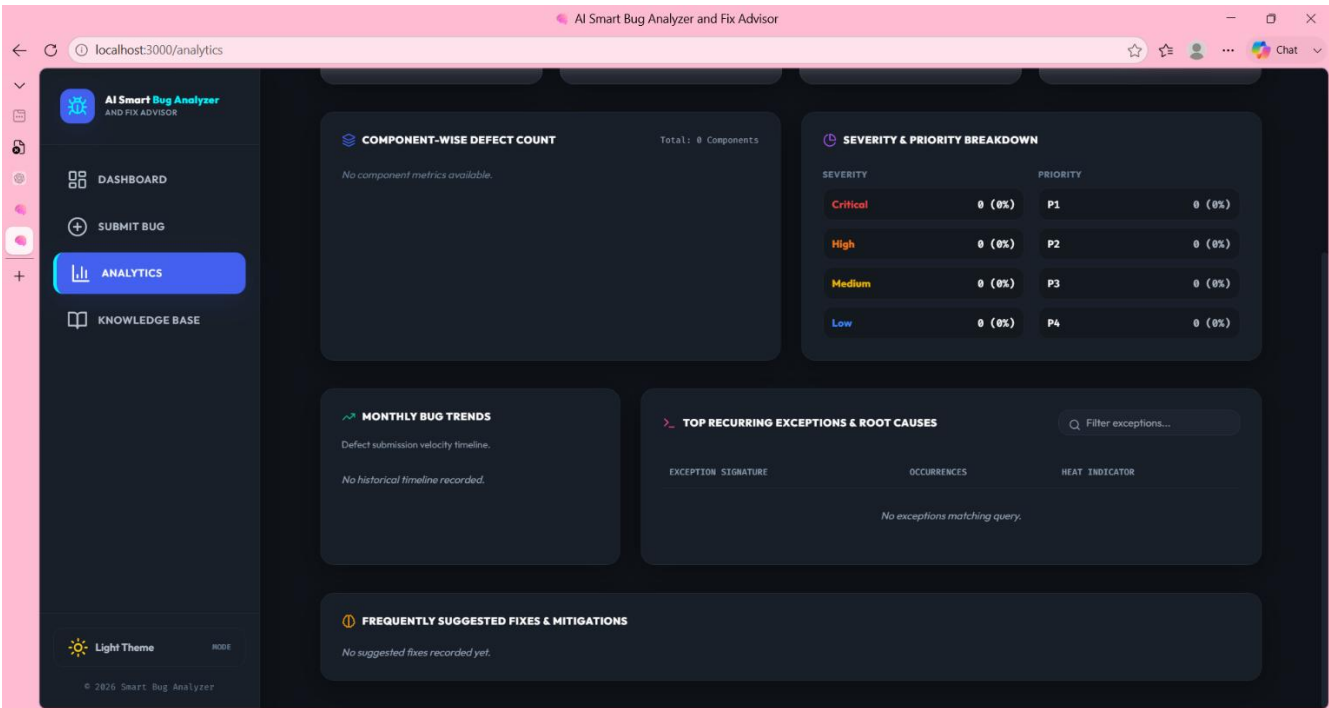


Figure 7: Root Cause and Fix Recommendation

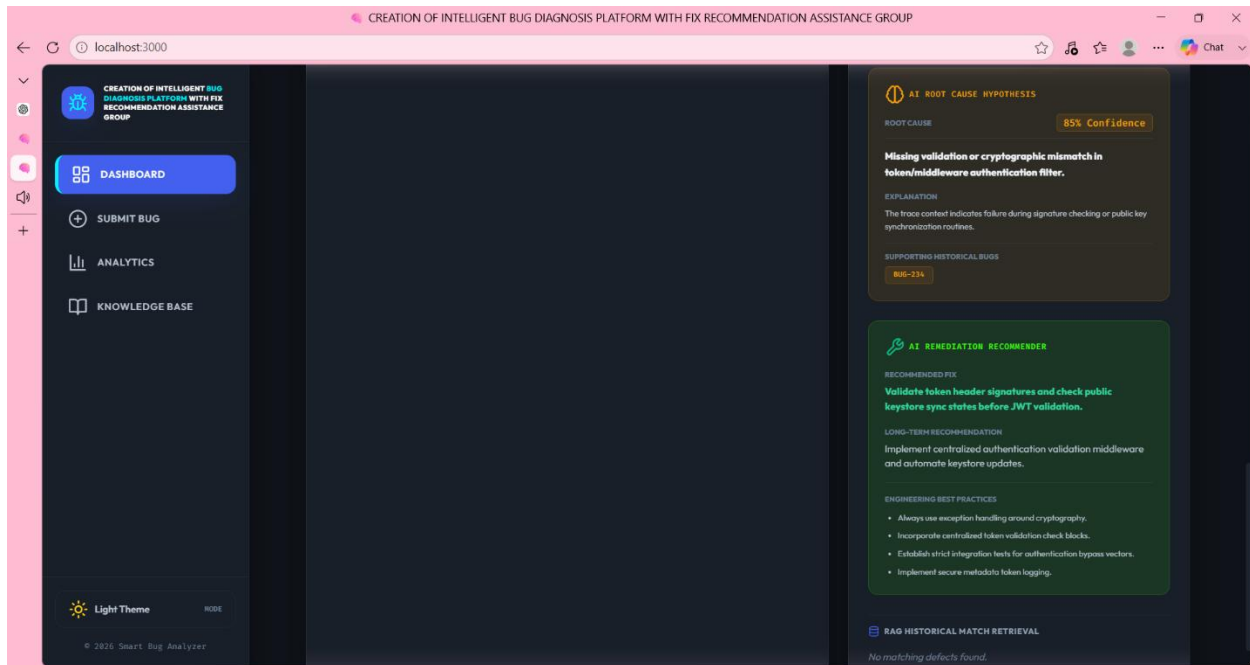


Figure 8: Resolution Verification Card

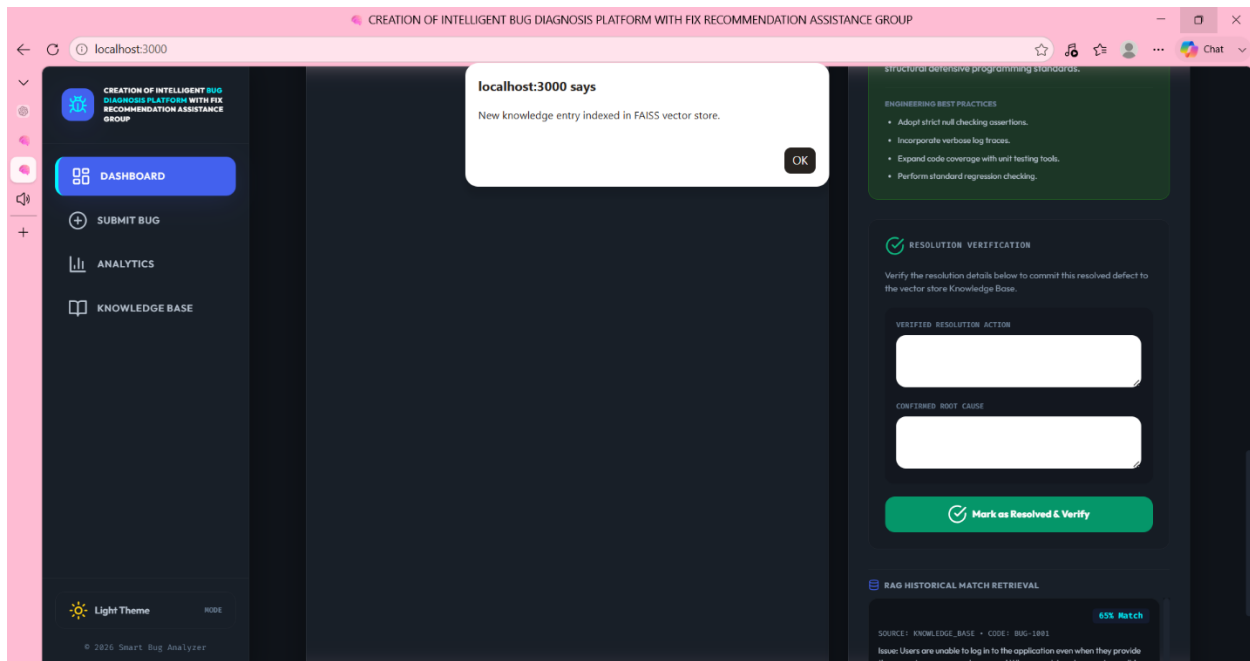
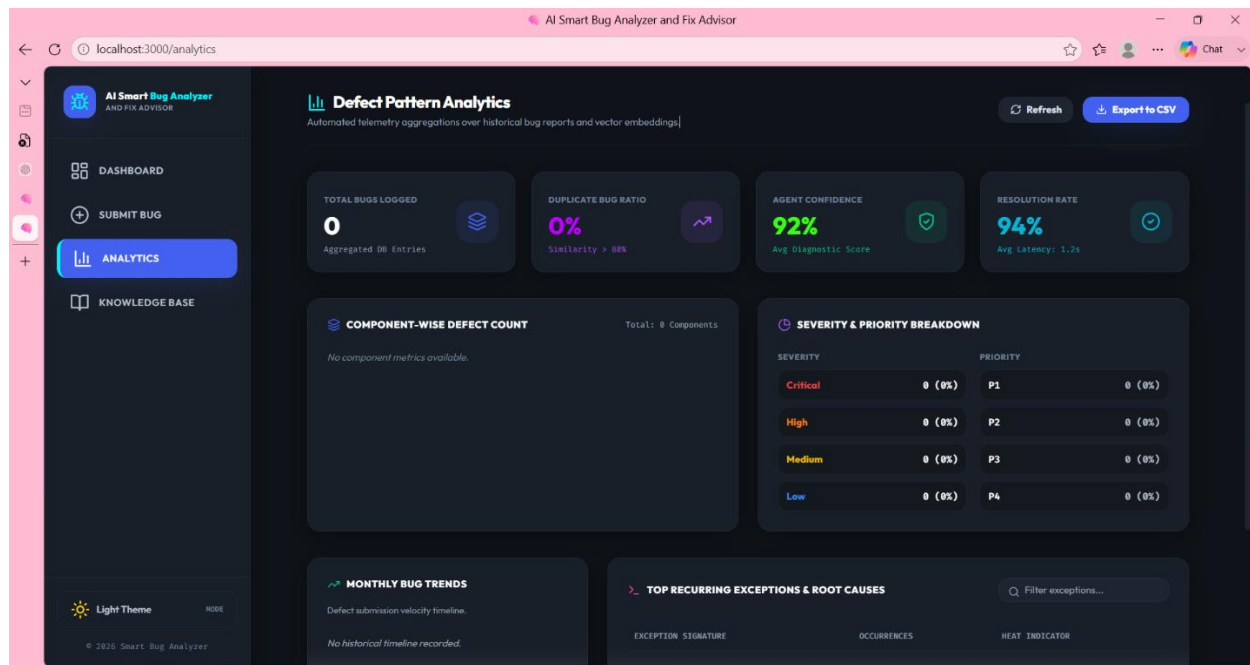
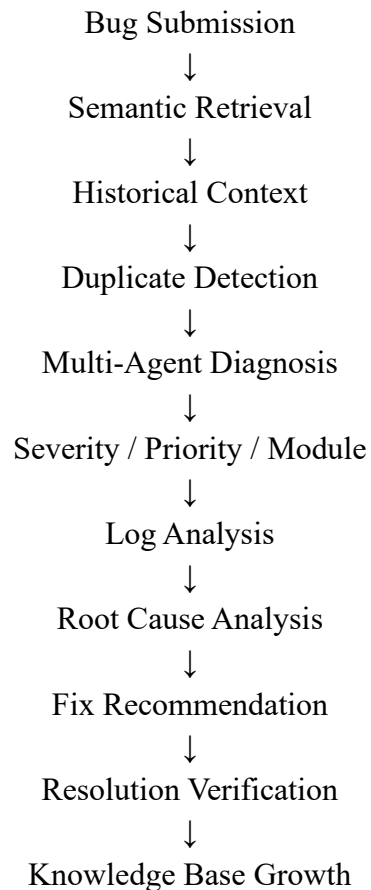


Figure 9: Defect Pattern Analytics Dashboard



APPENDIX B – PROJECT FLOW





Defect Pattern Analytics

APPENDIX C – KNOWLEDGE BASE GROWTH FLOW

Resolved Bug



Resolution Verification



Generate Embedding



FAISS Similarity Search



Similarity Comparison



Update Existing Knowledge / Create New Knowledge



FAISS Index Update

END OF REPORT